

Histogram Equalization for Grayscale and Color Images

María Romero Huertas. Image and Signal Processing

December 15, 2025

1 Problem statement

The goal of this assignment is to enhance image contrast using *histogram equalization*. I implemented global and local histogram equalization for grayscale images and extended the method to color images in the RGB color space, while preserving the overall color balance. All methods were implemented from scratch in MATLAB (without using built-in functions such as `histeq` or `adapthisteq`).

2 Mathematical background

If we consider an 8-bit grayscale image, we can represent its gray levels with r , where $r \in \{0, \dots, L-1\}$, $L = 256$. The size of the image in question will be $M \times N$.

In order to perform histogram equalization we need to calculate the PMF ($p(r_k)$) and the CDF ($c(r_k)$). The goal is to spread out the brightness range of the image to utilize it completely and increase the overall contrast of the picture. In order to do this, we calculate

$$p(r_k) = \frac{h(r_k)}{MN} \quad (1)$$

in order to obtain

$$c(r_k) = \sum_{j=0}^k p(r_j). \quad (2)$$

and then apply a transformation to map every pixel of value r_k to s_k , increasing the global contrast. [1]

$$s_k = T(r_k) = \text{round}((L-1)c(r_k)), \quad (3)$$

For the local (adaptive) histogram equalization, instead of calculating the histogram for the whole image and then applying the monotone transformation, we perform the mapping for the neighboring area around each pixel. This produces a better result in images where the brightness varies greatly depending on the region. However, this locality can amplify edges too strongly and also noise in smooth regions, so I decided to apply clipping before calculating the CDF to avoid the noise amplification problem (CLAHE). [2]

3 Selected methods

This project encompasses several approaches to histogram equalization. My final implementation includes global histogram equalization and a contrast-limited adaptive method using local windows, which was made more computationally efficient using the tile-based CLAHE algorithm with bilinear interpolation. [2]

I implemented image type conversions manually by explicitly scaling and casting between `uint8` and `double`, avoiding the use of `im2double` and `im2uint8` (from the Image Toolbox).

Global histogram equalization

Global HE uses the histogram of the entire image to map a monotone transformation based on the CDF, as explained in the previous section. This technique increases global contrast and is not computationally expensive, but it performs poorly when there are spatial variations in brightness.

Local adaptive histogram equalization

The local histogram equalization divides the picture into sections and computes the local histogram for each one. To prevent excessive contrast expansion in regions with little variation, I apply contrast limiting: each histogram bin is clipped to a fraction of the window size, and clipped counts are redistributed. This reduces noise amplification and produces smoother results. However, calculating a full histogram for every pixel is computationally expensive, so I introduced interpolation to optimize my function implementation.

Tile-based CLAHE with interpolation

To address the computational cost of per-pixel adaptive HE, I implemented the more efficient CLAHE variant with the algorithm described in [2], where the image is divided into an $n_y \times n_x$ grid of tiles (in this case 8×8). For each tile, I computed a clipped histogram equalization mapping. Then, I assigned values to the pixels between tile centers using bilinear interpolation of the four nearest tile mappings. For pixels located near image edges and corners, the interpolation was reduced to linear interpolation or nearest-neighbor mapping. This way I could preserve the smoothness across tile boundaries while greatly reducing the number of histogram computations. The result closely matches that of dense local HE, but at a significantly lower cost.

Color histogram equalization

To avoid color distortion, I applied the equalization to the value -V- channel in HSV (which manipulates the brightness of the image) instead of to each of the RGB channels separately. Then, to achieve a better result I introduced an α variable that acts as a blending factor, to control the strength of the image modification and preserve color balance when applying the contrast improvement equalization.

$$V_{new} = (1 - \alpha)V + \alpha V_{equalized} \quad (4)$$

I implemented the RGB–HSV and HSV–RGB transformations manually using standard chroma-based formulas [3].

4 Experimental results

For testing I applied my function `hist_equalization_demo` to the `lena-color` image, which is standard for image testing, and the `pout-tif` image from MATLAB to check performance on a low-contrast grayscale picture. In both cases we can see a contrast improvement. The global equalization produces a more dramatic result with a higher overall contrast, richer colors and stronger saturation thanks to the HSV method. Meanwhile, the local equalization reveals finer details and structure in the clothes, hair and skin of both test subjects. In either case the resulting images are an improvement over the originals, showing little to no distortion (thanks to the clipping to avoid excessive noise).



Figure 1: Grayscale results: original, global histogram equalization, and local histogram equalization with 1% clipping.

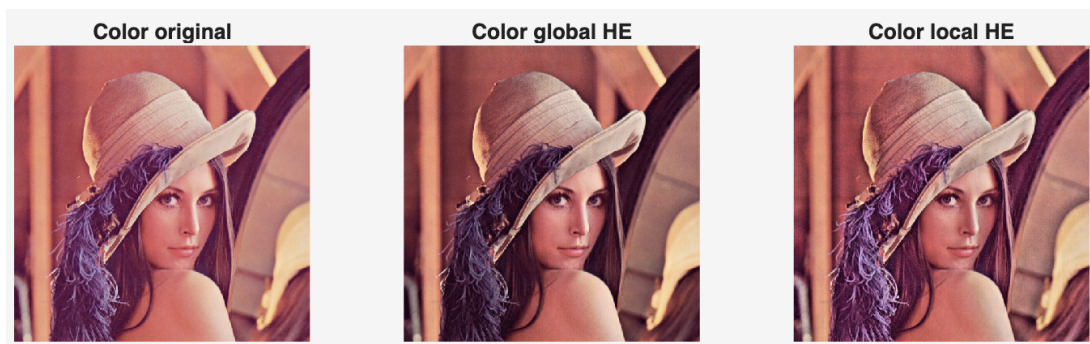


Figure 2: Color results: original RGB image, global histogram equalization, and local histogram equalization (HSV value channel) with 1% clipping.

References

- [1] “Histogram equalization,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Histogram_equalization (accessed 2024).
- [2] “Adaptive histogram equalization,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/Adaptive_histogram_equalization (accessed 2024).
- [3] “HSL and HSV,” *Wikipedia, The Free Encyclopedia*. Available at: https://en.wikipedia.org/wiki/HSL_and_HSV (accessed 2024).